

Lucida

An AI workforce for a small shop. One supervisor routes work to eight specialists — research, photos, pricing, stock, ads, customers, delivery, reporting — and they share one memory of the business. The owner talks to it in plain language, or sends a photo of what they make.

It is built for a shop in Dhaka: Bengali and Banglish messages, taka, Pathao and Steadfast couriers, weight-based delivery pricing, and a free model tier that has to survive a daily cap.

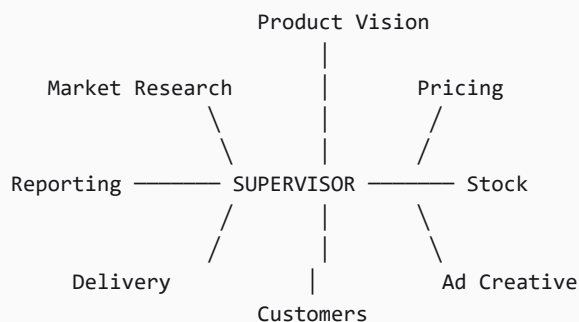
Contents

- [How it works](#)
 - [The screens](#)
 - [Where the data lives](#)
 - [Models, and what happens when one dies](#)
 - [Integrations](#)
 - [Running it](#)
 - [Configuration](#)
 - [Project layout](#)
 - [What is honest about the current state](#)
-

How it works

The star

A LangGraph state machine. Every request enters at the **supervisor**, which reads it, plans, and hands to one specialist. That specialist works, writes what it learned to shared memory, and returns to the supervisor, which picks the next one or finishes.



It is a star, not a pipeline: a question about stock does not drag six agents along with it. The supervisor decides, and the route it took is recorded.

SPECIALIST	WHAT IT OWNS
Market Research	What sells, what rivals charge, what people are asking for
Product Vision	Reads a photo and says what the product is and whether it is worth selling
Pricing	Unit cost, price, margin, break-even
Stock	Quantities, reorder levels, what runs out first
Ad Creative	Writes platform-specific copy and publishes it, after approval
Customers	Reads DMs and comments, sentiment, unmet demand, drafts replies
Delivery	Quotes a parcel and books the courier, after approval
Reporting	Writes the day up in the owner's own numbers

Gates

Anything irreversible or costly — publishing an ad, spending money, booking a courier — stops and waits. The run suspends at a LangGraph interrupt, the owner answers the card on **Home**, and approving resumes the graph from exactly that point rather than starting over.

Memory

Two halves, both per shop:

- **Structured** — SQLite, 24 tables: products, inventory, orders, pricing history, social messages, campaigns, deliveries, chat threads, competitors.
- **Semantic** — a small vector store of what agents learned, so later runs recall earlier conclusions.

Watching it happen

`/api/events` holds one connection open and pushes each step as it is recorded. The Workforce screen draws the run live: a card lights when its agent starts, handoffs appear as numbered arrows in order, and every step scrolls into a feed with its timestamp and cost.

The screens

Six, and the rail holds exactly these six.

SCREEN	WHAT IT IS FOR
Home /	Where the shop stands — sales, stock cover, promises — and anything waiting on you. An approval the workforce is suspended on is answered here, and nothing else moves until it is.
Chat /chat	Ask anything. Conversations are kept and can be reopened. Send a photo and Product Vision reads it. A research question asks <i>where</i> to look before spending anything. The ad studio opens from here.
Products /products	What you sell, what it costs, and what it weighs . Weight is what every delivery quote is worked out from, so delivery pricing opens from here.
Customers /customers	Both sides of every conversation the bot held on your behalf, and the reviews it asked for afterwards.
Workforce /workforce	The graph, live. Click a specialist to hand it a job directly, over the supervisor's head. Below it, the operator's view: recent runs, what the team wrote down, and what the model calls cost.
Settings /settings	Channels and couriers, and your own account. Credentials are checked against the live API before they are stored.
Service admin /admin	Operators only. Every shop on the installation and what it is using.

Anything that is a step inside a job rather than a place you set out for is reached from the screen that owns it. `/studio` and `/delivery` are still real addresses; they are simply not competing for a slot in the rail.

Earlier builds shipped four overlapping interfaces — a Streamlit workspace, these server-rendered pages, and two React design mockups. The mockups drew real figures behind unwired placeholder buttons, and one of them was the first entry in the rail, so the most likely first click in the app landed somewhere nothing happened. They have been removed; `/board`, `/connect` and `/account` redirect to whichever screen took the work over.

Where the data lives

```
data/
├─ accounts.db      every owner: sign-in name, bcrypt hash, business
├─ lucida.db        the trace – every step every agent has taken
├─ checkpoints.db   LangGraph state, so a paused run can resume
├─ session.key      cookie signing key (gitignored)
└─ shops/
    └─ <account-id>/
        ├─ shop.db      this shop's business, alone
        └─ knowledge.jsonl this shop's semantic memory
```

Isolation is by file, not by a column. Every agent shares one `memory` singleton; the web layer rebinds it to the signed-in owner's directory before any agent runs. No query has to remember a `WHERE owner_id = ?`, and a forgotten filter cannot leak one shop into another.

The trace is the exception — one file for the whole machine — so session ids carry a hash of the owner and the dashboard matches on that prefix.

Models, and what happens when one dies

Free tiers are rationed and providers retire models without warning. Both have happened during this project, so the chain is built for it:

```
your model → the other Groq models → Google (if a key is set)
```

A model moves the chain along when it is **capped** (429), **retired** (404), or **returns the right JSON but not as a tool call** — a real intermittent failure that used to kill a whole run. Only when everything is exhausted does the owner see a message, and it says when to come back rather than showing a provider traceback.

Photo reading runs on Google, because Groq serves no multimodal model.

Integrations

	STATE
Groq	Text. Free tier, daily caps per model.
Google AI Studio	Photo reading, and the fallback when Groq is spent. Free.
Pathao	Live. Books real parcels and prices them from Pathao's own rate card.
Steadfast	Ready — paste the key pair from your merchant portal.
DuckDuckGo	Web search, no key, searches your own country's index.
Tavily	Better search if a key is set — returns page content, not one-line snippets.
Pollinations	Ad artwork, no key. Free Gemini keys get no image quota, so this is the default.
Telegram	Live. A real customer messages the bot; the Customers agent reads and classifies it, and the owner replies from the browser. No review process, no domain.
Messenger / Facebook / Instagram	Built and tested against the Graph API. Needs a Meta app, App Review <i>and</i> Business Verification before it reaches real customers.
YouTube	Needs an OAuth client. An API key cannot upload.
Email	Not used. Sign-in is a name and a password; there is no verification step.

Anything not connected runs against a **simulated** adapter that is labelled as such everywhere it appears, and nothing simulated is ever written to the shop's records as though a customer sent it.

Running it

```
python -m venv .venv
.venv\Scripts\pip install -r requirements.txt      # Windows
# .venv/bin/pip install -r requirements.txt        # macOS / Linux

cp .env.example .env          # then add GOOGLE_API_KEY
python serve.py
```

Then open **http://127.0.0.1:8000**.

Tests — 69 of them, none needing an API key:

```
python -m pytest tests -q
```

Deployment — persistent disk, one instance, long request timeouts — is in [DEPLOY.md](#).

Configuration

Everything lives in `.env`, which is gitignored.

SETTING	FOR
GOOGLE_API_KEY	The text models and photo understanding. Required.
GROQ_API_KEY	Alternative text provider (<code>AIW_PROVIDER=groq</code>). No vision model.
TELEGRAM_BOT_TOKEN	The live customer channel. From <code>@BotFather</code> .
GOOGLE_API_KEY	Photo reading, and the cross-provider fallback.
AIW_SECRET_KEY	Signs session cookies. Generated and kept if unset.
AIW_SMTP_*	Unused — kept for a future password-reset flow.
TAVILY_API_KEY	Better web search.
META_APP_ID / META_APP_SECRET	Turns Connect into one-click sign-in.
YOUTUBE_CLIENT_ID / _SECRET	YouTube uploads.
LUCIDA_ADMIN_EMAILS	Who sees <code>/admin</code> . Never a database column.
LUCIDA_DATA_DIR	Where the data lives. Point at a volume in production.

Courier and social credentials are entered **in the app**, not here, and are stored per shop.

Project layout

serve.py	every route; the only place the web talks to the graph
src/lucida/	
graph.py	the state machine, and the runtime that drives it
supervisor.py	planning and routing
state.py	what travels between agents
llm.py	model construction and the failover chain
agents/	the eight specialists, one file each
memory/	db.py (SQLite), vector.py (semantic), shared.py (both)
tools/	search, images, couriers, channels, delivery pricing, connections, currency, sandboxed calculation
observability.py	the event bus every screen reads from
web/	
app_ui.py	every signed-in screen, and the rail they share
screens.py	sign-in, sign-up, email confirmation
admin.py	the operator's view
bridge.py	shop data → the shapes the screens render
auth.py	accounts, bcrypt, Google linking
static/	the favicon; there are no other static assets

What is honest about the current state

Working and verified end to end: chat with history, photo → vision, research with a source picker, ad generation and editing, product management, delivery quoting and booking against Pathao's live API, the live workforce graph, task assignment, and the admin view.

Waiting on credentials, not code: Meta (app + App Review), YouTube (OAuth client), Steadfast (key pair), email (SMTP).

Known weak: ad artwork, while it runs on the free keyless model — it gets the colour and the setting right and often invents the object. Uploading your own photo is the reliable path.

Deliberately absent: the admin view reads counts, never contents. It can tell you a shop has forty messages; it cannot show you one. That line is drawn in the module and printed on the page.